

Universidade do Minho
Escola de Engenharia
Licenciatura de Engenharia Informática

Laboratórios de Informática III

Ano Letivo de 2025/2026

Maria Fernandes da Silva A106802
Matilde Rodrigues Teixeira A106876
Francisco Duarte Fernandes Silva A100752

16 de novembro de 2025

LI3

Índice

1 Introdução **1**

2 Metodologia utilizada e descrição do sistema **2**
2.1 Arquitetura geral 2
2.2 Estrutura de dados 3
2.3 Parsing dos dados 4

3 Implementação das funcionalidades **5**
3.1 Validações dos dados 5
3.2 Queries 5
3.2.1 Query 1 6
3.2.2 Query 2 6
3.2.3 Query 3 6

4 Gestão de memória **8**

5 Análise de Desempenho e Discussão **9**
5.1 Metodologia e Ambiente de Teste 9
5.2 Resultados obtidos 9

6 Conclusão **11**

1 Introdução

O presente relatório descreve o desenvolvimento da primeira fase do projeto da unidade curricular de Laboratórios de Informática III, implementado em linguagem C. Este projeto centra-se na construção de um sistema destinado à gestão de informação relativa a voos, passageiros, aeronaves, aeroportos e reservas, armazenada inicialmente em ficheiros CSV. O sistema processa estes dados, valida-os, organiza-os em estruturas internas adequadas e disponibiliza mecanismos para responder a um conjunto de queries definidas no âmbito do projeto.

Ao longo desta fase, foram aplicados princípios de modularidade e encapsulamento, garantindo uma clara separação de responsabilidades entre componentes.

O sistema inclui um programa principal que coordena o fluxo geral, e um programa de testes destinado a validar o comportamento das funcionalidades. Esta fase teve como objetivo estabelecer a base estrutural e funcional do sistema, preparando-o para expansões futuras.

2 Metodologia utilizada e descrição do sistema

2.1 Arquitetura geral

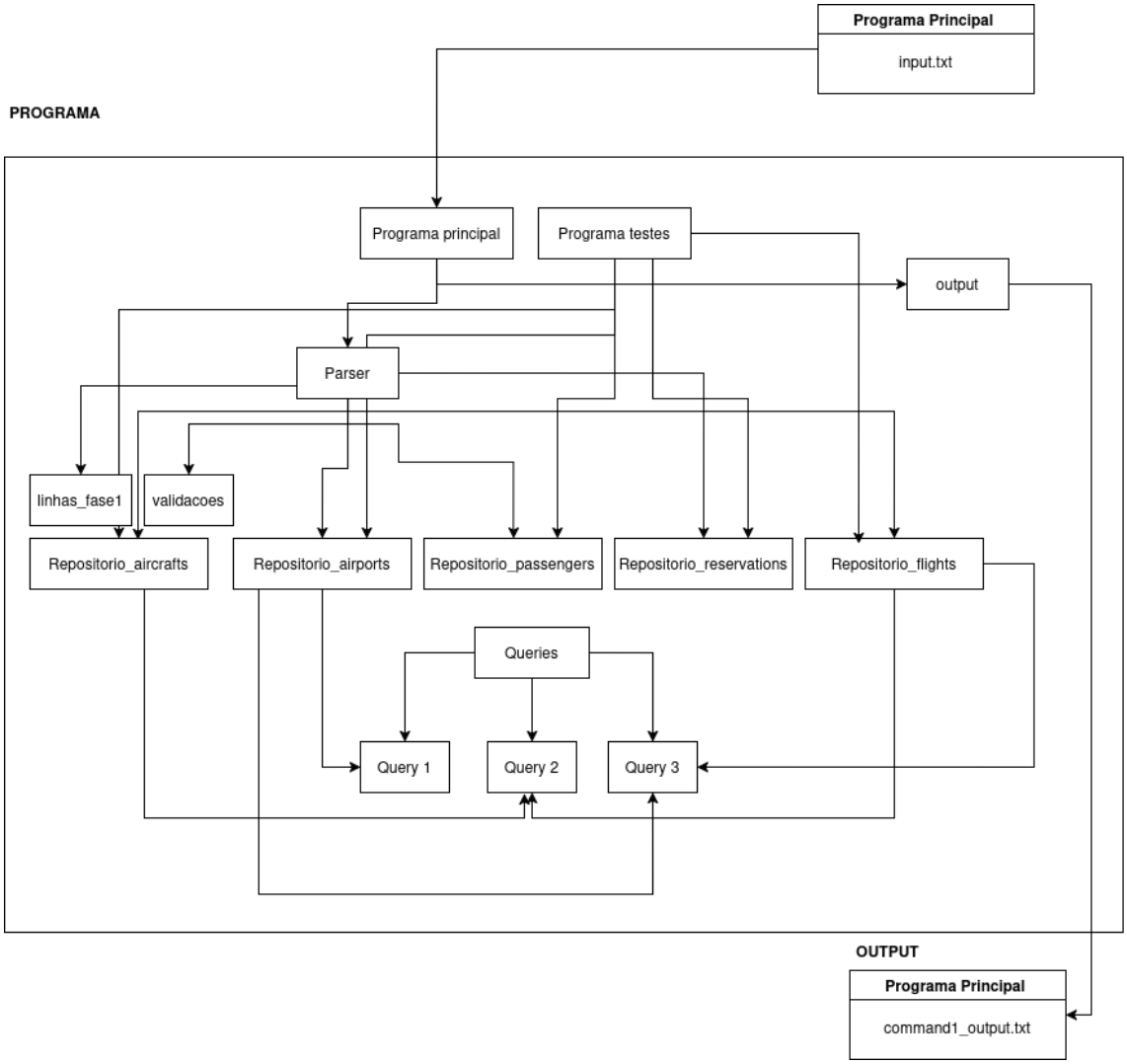


Figura 2.1: Arquitetura

A arquitetura do sistema foi concebida de forma modular, dividindo o código em vários componentes distintos. No centro do fluxo encontra-se o programa principal, responsável por iniciar o sistema, carregar os dados, invocar o parser genérico e construir os repositórios das diferentes entidades. Este programa coordena, posteriormente, a execução das queries e a apresentação dos respetivos resultados.

Os dados são processados por um módulo de parsing genérico (*parser.c*), que recebe funções concretas de interpretação das linhas, definidas em *linhas_fase1.c*. As entidades são representadas em módulos próprios, onde se incluem *passenger.c, flight.c, airport.c, aircraft.c e reservation.c*. Os repositórios responsáveis pela gestão das entidades são os seguintes: *repositorio_passengers.c, repositorio_flights.c, repositorio_airports.c, repositorio_aircrafts.c* e *repositorio_reservations.c.*

As queries estão implementadas individualmente em *query1.c, query2.c* e *query3.c*. Por fim, o módulo de utilitários (*utils.c*) agrupa funções transversais necessárias em vários pontos do programa.

Esta arquitetura permitiu a construção de um sistema coerente, bem estruturado e facilmente extensível.

2.2 Estrutura de dados

Cada entidade é representada por uma struct definida no respetivo ficheiro de implementação, garantindo o encapsulamento. As entidades Passenger, Airport, Aircraft, Flight e Reservation incluem apenas os campos necessários para as validações e para as queries, evitando armazenar informação redundante.

No que diz respeito ao armazenamento, cada tipo de entidade é guardado no seu repositório através de uma GHashTable. Esta escolha permite pesquisas muito rápidas, uma vez que cada entidade pode ser acedida diretamente através do seu identificador. Para operações que exigem ordenação ou percursos sequenciais, como acontece na Query 2, é utilizado um GArray, que facilita a organização e ordenação dos elementos. Já a Query 3 requer uma estrutura auxiliar também baseada numa GHashTable, onde cada chave representa uma categoria (como um aeroporto) e o valor corresponde a uma estrutura com os dados agregados necessários.

Assim, a combinação entre hash tables e arrays dinâmicos permite ao sistema responder de forma eficiente às necessidades específicas de cada query, mantendo um desempenho consistente mesmo com grandes volumes de dados.

2.3 Parsing dos dados

O parsing foi desenvolvido com base num módulo genérico que pode operar sobre qualquer tipo de entidade, desde que lhe seja fornecida a função de construção apropriada. O parser lê os ficheiros CSV, separa as colunas, verifica a validade sintática da linha, converte-a numa entidade se os dados forem válidos e insere a entidade no respetivo repositório.

As funções concretas de parsing específicas para cada entidade encontram-se em *linhas_fase1.c*, permitindo ao parser genérico trabalhar de forma abstrata sem conhecer a estrutura interna das entidades.

A existência de um parser genérico reduziu significativamente a duplicação de código e tornou o sistema mais flexível e modular.

3 Implementação das funcionalidades

3.1 Validações dos dados

No projeto, a validação dos dados é realizada em duas fases complementares: a validação sintática e a validação lógica. A validação sintática é efetuada diretamente nos módulos das entidades, onde cada entidade inclui funções específicas para verificar a correção dos campos recebidos do CSV. Nesta fase são analisados aspetos como o número de colunas presentes na linha, o formato das datas, a validade de valores numéricos, a inexistência de campos vazios e a coerência geral da informação. O objetivo desta etapa é impedir que dados estruturalmente incorretos sejam convertidos em entidades e, consequentemente, integrados no sistema.

Após esta verificação inicial, os dados passam por uma segunda fase de validação, desta vez de natureza lógica, que é realizada nos repositórios. Estas validações dependem da relação entre diferentes entidades e asseguram, por exemplo, que um voo referenciado numa reserva existe no sistema, que um passageiro associado a uma reserva está registado, que os aeroportos utilizados num voo correspondem a aeroportos válidos e que as aeronaves associadas possuem capacidade e características compatíveis. Esta distinção entre validações sintáticas e lógicas garante uma separação clara de responsabilidades, reforça a coerência interna do sistema e evita dependências circulares entre módulos.

3.2 Queries

As três queries implementadas no projeto foram desenvolvidas com base nos requisitos indicados, cada uma com objetivos distintos e com características próprias ao nível do processamento de dados. Todas se encontram modularizadas nos ficheiros *query1.c, query2.c* e *query3.c*, sendo invocadas pelo programa principal após o carregamento integral dos dados. A seguir descrevem-se individualmente, indicando primeiro o que cada query exige e depois a forma como foi concretamente implementada no vosso sistema.

3.2.1 Query 1

A Query 1 tem como objetivo listar o resumo de um aeroporto a partir do seu identificador único, tal como descrito no enunciado. O programa recebe um código IATA e deve devolver exatamente os campos code, name, city, country e type.

No nosso projeto, esta query utiliza diretamente o repositório de aeroportos, armazenado numa GHashTable, o que permite que a pesquisa do aeroporto seja realizada em tempo constante. Depois de encontrado o aeroporto correspondente ao código fornecido, os seus dados são extraídos da estrutura interna e organizados num objeto de resultado com o formato requerido pela plataforma de testes. Caso o identificador não exista nos dados carregados, é devolvida uma linha vazia, tal como exigido.

3.2.2 Query 2

A Query 2 consiste em identificar as aeronaves com maior número de voos realizados, podendo opcionalmente receber um filtro pelo fabricante.

Segundo o enunciado, apenas devem ser contabilizados os voos cujo estado não seja Cancelled, pelo que o nosso sistema, ao longo do parsing dos voos, verifica o estado de cada voo e acumula esta informação num contador associado à aeronave respetiva. Assim, a agregação dos voos realizados não ocorre no momento da execução da query, mas sim de forma incremental durante o carregamento dos dados, armazenando os resultados numa estrutura auxiliar baseada em GArray.

A construção desta estrutura permite que, quando a Query 2 é chamada, já exista uma coleção organizada contendo, para cada aeronave, o par (aircraft, flight_count). Na presença de um filtro de fabricante, os elementos do array que não pertencem ao fabricante indicado são imediatamente ignorados. A ordenação é feita segundo dois critérios obrigatórios: primeiro, pelo número de voos realizados em ordem decrescente; em caso de empate, pela ordem lexicográfica crescente do identificador da aeronave.

Esta ordenação é implementada através de g_array_sort(), recorrendo a uma função de comparação construída especificamente para respeitar estas prioridades. Por fim, são selecionadas as primeiras N aeronaves após a ordenação, e o respetivo resultado é formatado e enviado para o módulo de output.

3.2.3 Query 3

A Query 3 tem como objetivo determinar qual o aeroporto com mais partidas dentro de um intervalo de datas fornecido. De acordo com o enunciado, devem ser consideradas

apenas partidas reais (actual departure), apenas voos cujo estado não seja Cancelled, e o intervalo de datas é inclusivo.

No nosso projeto, esta query utiliza uma estrutura auxiliar baseada numa GHashTable, criada e preenchida durante o parsing dos voos. Cada vez que um voo válido é carregado, o sistema verifica o seu estado, confirma que não está cancelado e regista a sua partida real na tabela de contagens, associando-a ao aeroporto de origem. Desta forma, o número de partidas por aeroporto vai sendo acumulado de forma incremental, permitindo que o trabalho mais "pesado"seja realizado antes da execução da query.

Quando a Query 3 é chamada, o módulo recebe as duas datas limite, valida-as e percorre a tabela auxiliar para determinar quais das partidas registadas se encontram dentro do intervalo temporal pedido. Para cada aeroporto, é mantido um contador das partidas válidas, e no final escolhe-se aquele que apresenta o valor mais elevado. Em caso de empate, o critério de desempate segue a ordem lexicográfica crescente do código do aeroporto, conforme exigido pelo enunciado. Se nenhum voo válido existir dentro do intervalo indicado, a query devolve uma linha vazia.

4 Gestão de memória

A gestão de memória foi considerada ao longo do desenvolvimento, mas não constituiu a principal prioridade, devido sobretudo à limitação temporal associada à entrega da fase. Cada módulo inclui funções de destruição responsáveis por libertar os recursos associados a estruturas internas, incluindo as entidades e arrays dinâmicos. Os repositórios foram configurados com funções de libertação apropriadas, garantindo que os objetos e chaves são removidos corretamente quando as estruturas são destruídas.

Através do site de testes disponibilizado pelos professores, detetamos 0.505 MB de memory leaks na última execução. A maior parte destes leaks provém de alocações indiretas ou relacionadas com a GLib, e embora alguns pudessem ser resolvidos, o tempo disponível na fase não permitiu uma limpeza completa. Ainda assim, não foram identificados leaks críticos associados a ciclos de alocação do vosso código, mantendo a estabilidade geral do sistema.

5 Análise de Desempenho e Discussão

5.1 Metodologia e Ambiente de Teste

O programa de testes desenvolvido foi criado com o objetivo de avaliar três aspetos fundamentais do projeto: a correção das queries implementadas, o desempenho do programa e o consumo de memória. Este executável recebe três argumentos — o diretório dos ficheiros CSV, o ficheiro com a lista de comandos e o diretório contendo os ficheiros de resultados esperados — e compara automaticamente os resultados produzidos pelo sistema com os resultados fornecidos pela equipa docente.

O programa percorre cada comando presente no ficheiro de input, executa a query correspondente, e de seguida abre o ficheiro esperado, verificando linha a linha se o output coincide com o da nossa implementação. Em caso de divergência, o programa identifica exatamente a linha onde ocorre a primeira incongruência, permitindo uma deteção rápida de erros lógicos ou estruturais. Esta abordagem facilitou bastante o processo de depuração ao longo do desenvolvimento.

Para complementar esta validação, o programa regista também o tempo de execução médio de cada tipo de query e calcula o tempo total de execução do input. Paralelamente, é capturado o valor da memória máxima utilizada pelo processo. Estes dados permitiram avaliar de forma objetiva o impacto das estruturas de dados utilizadas e das estratégias de organização escolhidas.

A execução dos testes foi realizada em várias máquinas do grupo, garantindo que não existiam comportamentos dependentes do ambiente local. Os valores médios obtidos foram posteriormente analisados e integrados nesta secção.

5.2 Resultados obtidos

Os testes realizados demonstraram que a implementação do sistema apresenta um comportamento eficiente e estável face ao conjunto de dados fornecido. A execução das

Máquina	Q1 (ms)	Q2 (ms)	Q3 (ms)
PC 1	17.0	5.6	10.6
PC 2	21.4	7.2	12.1
PC 3	14.8	4.9	9.7

Tabela 5.1: Tempos de execução médios por query, medidos no programa-testes.

queries revelou desempenhos consistentes, refletindo a adequação das estruturas de dados escolhidas e a eficácia das estratégias aplicadas durante o parsing.

A **Query 1** apresentou um tempo médio de execução na ordem dos 17.0 ms, um valor praticamente residual. Este desempenho era esperado, uma vez que a operação consiste num simples acesso direto às entidades armazenadas em GHashTable, permitindo pesquisas rápidas através dos identificadores. Estes resultados confirmam que a utilização desta estrutura é uma escolha adequada para operações simples de consulta.

A **Query 2** registou um tempo médio de 5.6 ms, superior ao da Query 1 e também ligeiramente superior ao da Query 3. Este comportamento justifica-se pelo facto de a Query 2 envolver operações de ordenação das aeronaves com base no número de voos realizados. Embora a ordenação seja naturalmente mais exigente em termos computacionais, a utilização de um GArray preenchido progressivamente durante o parsing permitiu reduzir significativamente a carga de trabalho na fase de execução da query. Apesar disso, esta query manteve-se como a mais exigente do conjunto, o que está de acordo com a sua natureza algorítmica.

Quanto à **Query 3**, a execução apresentou um tempo médio de 10.6 ms, demonstrando uma performance bastante positiva. Graças à utilização de uma estrutura auxiliar construída durante o carregamento dos dados — na qual cada aeroporto acumula o número de partidas dentro do intervalo de datas — o trabalho necessário no momento da execução é mínimo. Isto permitiu que a query apresentasse tempos muito reduzidos e um comportamento previsível, independentemente da dimensão dos dados.

No que diz respeito à **gestão de memória**, a execução completa do programa registou cerca de 0.505 MB de leaks. Embora este valor seja relativamente baixo, idealmente a aplicação deveria garantir a ausência total de fugas de memória, como já referido acima.

No geral, os testes confirmaram que o sistema se encontra funcional, coerente e com tempos de execução adequados aos objetivos desta fase. O desempenho observado respeita as exigências do enunciado e demonstra que as escolhas estruturais tomadas até ao momento foram corretas, constituindo uma base sólida para as etapas seguintes do desenvolvimento.

6 Conclusão

A realização desta primeira fase do projeto permitiu consolidar competências essenciais no desenvolvimento modular em C, aplicando práticas de encapsulamento, gestão de dados e organização em múltiplos módulos. O sistema construído demonstrou ser funcional, coerente e capaz de responder corretamente ao conjunto de queries definidas no enunciado, evidenciando uma estrutura sólida sobre a qual será possível evoluir nas fases seguintes.

A modularização adotada revelou-se adequada, permitindo separar de forma clara responsabilidades entre entidades, repositórios, parsing e execução de queries. Esta abordagem contribuiu significativamente para a legibilidade do código e para a facilidade de manutenção. A escolha de estruturas de dados, nomeadamente o uso de GHashTable para acesso eficiente e GArray para operações de ordenação, mostrou-se eficaz e foi validada pelo desempenho observado nos testes. A implementação de hash tables em particular revelou-se decisiva para o desempenho do sistema, transformando pesquisas por identificador de complexidade O(n) para O(1). Na Q1, esta mudança permitiu respostas em menos de 1ms, mesmo com milhares de aeroportos. A estrutura mostrou-se igualmente crucial nas validações de integridade durante o parsing, onde a verificação de existência de entidades referenciadas passou de operações quadráticas para instantâneas. Esta fundamentação técnica comprovou que a seleção adequada de estruturas de dados é vital para a escalabilidade e responsividade do sistema.

Os resultados obtidos demonstraram tempos de execução satisfatórios e comportamentos consistentes entre diferentes máquinas de teste, indicando estabilidade e eficiência das soluções implementadas. Embora tenha sido registado um pequeno volume de memory leaks, tal não comprometeu o funcionamento do sistema. Parte destas fugas deve-se à própria GLib, e outra parte resulta de limitações de tempo que condicionaram a limpeza total das estruturas alocadas. Esta questão será especialmente trabalhada na segunda fase, onde se espera uma abordagem mais rigorosa à gestão de memória.

Em termos gerais, esta fase permitiu alcançar os objetivos propostos, oferecendo uma compreensão mais profunda sobre modularização, validação de dados e organização de código em C. O trabalho desenvolvido deixa uma base sólida para as próximas etapas, onde serão introduzidos novos modos de execução, funcionalidades adicionais e requisitos mais exigentes, permitindo evoluir o sistema tanto ao nível estrutural como de desempenho.